Lab 10

K-means, K-Medoids

Step 1: Import Libraries

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load the Dataset

In [ ]:
```
df = pd.read_csv("StudentsPerformance.csv")
```

Step 3: Data Overview

In [ ]:
```
df.isnull().count()
```

Step 4: Display PairPlot

In [ ]:
```
sns.pairplot(df)
plt.show()
```

Step 5: Correlation heatmap

In [ ]:
```
corr = df.corr(numeric_only=True)
```

In [ ]:
```
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()
```

Step 6: Distribution of numerical features

In [ ]:
```
numeric_cols = df.select_dtypes(include=['number']).columns
```

In [ ]:
```
n_rows = (len(numeric_cols) + 1) // 2
plt.figure(figsize=(12, n_rows * 4))
```

In [ ]:
```
for i, col in enumerate(numeric_cols):
    plt.subplot(n_rows, 2, i + 1)
    sns.histplot(df[col], kde=True, color='skyblue')
    plt.title(f'Distribution of {col}')
```

```
plt.tight_layout()
plt.show()
```

Step 7: Apply StandardScaler

In [ ]:
```
from sklearn.preprocessing import StandardScaler
```

In [ ]:
```
scaler = StandardScaler()
```

In [ ]:
```
scaled_data = scaler.fit_transform(df[numeric_cols])
```

In [ ]:
```
df_scaled = pd.DataFrame(scaled_data, columns=numeric_cols)
```

Step 8: Elbow method to find optimal k

In [ ]:
```
from sklearn.cluster import KMeans
```

In [ ]:
```
inertia = []
k_range = range(1, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(df_scaled)
    inertia.append(kmeans.inertia_)
```

In [ ]:
```
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, marker='o', linestyle='--', color='b')

plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.xticks(k_range)
plt.grid(True)
plt.show()
```

Step 9:Based on the elbow plot, choose an appropriate k value (e.g., k=3)

In [ ]:
```
kmeans_final = KMeans(n_clusters=3, random_state=42, n_init=10)

cluster_labels = kmeans_final.fit_transform(df_scaled)
clusters = kmeans_final.predict(df_scaled)
```

In [ ]:
```
df['Cluster'] = clusters
```

Step 10: Print Cluster Center

In [ ]:
```
centers_scaled = kmeans_final.cluster_centers_
```

In [ ]:
```
centers_original = scaler.inverse_transform(centers_scaled)
```

In [ ]:
```
df_centers = pd.DataFrame(centers_original, columns=numeric_cols)
```

In [ ]:
```
df_centers.index.name = 'Cluster'
df_centers = df_centers.reset_index()
print(df_centers)
```

Step 11: Plot Cluster

In [ ]:
```
x_axis = numeric_cols[0]
y_axis = numeric_cols[1]
```

In [ ]:
```
plt.figure(figsize=(10, 6))
scatter = plt.scatter(df[x_axis], df[y_axis], c=df['Cluster'], cmap='viridis', s
```

In [ ]:
```
x_idx = list(numeric_cols).index(x_axis)
y_idx = list(numeric_cols).index(y_axis)
```

In [ ]:
```
plt.scatter(centers_original[:, x_idx], centers_original[:, y_idx],
            s=250, c='red', marker='X', label='Centroids')
```

In [ ]:
```
plt.title(f'Cluster Visualization: {x_axis} vs {y_axis}', fontsize=15)
plt.xlabel(x_axis)
plt.ylabel(y_axis)
plt.legend(*scatter.legend_elements(), title="Clusters", loc='upper right')
plt.grid(True, linestyle='--', alpha=0.5)

plt.show()
```

Step 12:Analyze clusters

In [ ]:
```
cluster_analysis = df.groupby('Cluster').mean(numeric_only=True)
cluster_analysis['Count'] = df.groupby('Cluster').size()
print(cluster_analysis)
```

Step 13: Perform K-Medoids

In [ ]:
```
from sklearn_extra.cluster import KMedoids
```

In [ ]:
```
k_medoids = KMedoids(n_clusters=3, random_state=42, metric='manhattan')
```

In [ ]:
```
k_medoids.fit(df_scaled)
```

In [ ]:
```
medoid_labels = k_medoids.labels_
```

In [ ]:
```
df['KMedoids_Cluster'] = medoid_labels
```

In [ ]:
```
medoids_indices = k_medoids.medoid_indices_
representative_points = df.iloc[medoids_indices]
```

In [ ]:
```
print(representative_points)
```

Step:14 Comparison of K-means and K-medoids Clusters

In [ ]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6), sharey=True)
```

In [ ]:
```
x_axis = numeric_cols[0]
y_axis = numeric_cols[1]
```

In [ ]:
```
scatter1 = ax1.scatter(df[x_axis], df[y_axis], c=df['Cluster'], cmap='viridis',
ax1.set_title('K-Means Clustering', fontsize=14)
ax1.set_xlabel(x_axis)
ax1.set_ylabel(y_axis)
```

In [ ]:
```
scatter2 = ax2.scatter(df[x_axis], df[y_axis], c=df['KMedoids_Cluster'], cmap='p
ax2.set_title('K-Medoids Clustering', fontsize=14)
ax2.set_xlabel(x_axis)
```

In [ ]:
```
ax1.legend(*scatter1.legend_elements(), title="Clusters")
ax2.legend(*scatter2.legend_elements(), title="Clusters")

plt.tight_layout()
plt.show()
```

Step: 15 | USE KMEAN++

In [ ]:
```
from sklearn.cluster import KMeans
```

In [ ]:
```
kmeans_plus = KMeans(n_clusters=3, init='k-means++', random_state=42, n_init=10)
```

In [ ]:
```
kmeans_plus.fit(df_scaled)
```

In [ ]:
```
df['Cluster_KPlus'] = kmeans_plus.labels_
```